# Cyber Security Internship Report

Redynox

Intern: Bithin Krishna Radhakrishnan

Batch: [to be added]

Date: October 12, 2026

# 1. Introduction

During this internship at Redynox I worked on two technical tasks. Task 1 introduced the basics of network security: common threats, basic protections on a home network, and traffic analysis with Wireshark. Task 2 introduced web application security, where I found and exploited common vulnerabilities in WebGoat, an intentionally insecure application I ran locally.

The purpose of the work was to learn how threats work and how they are prevented. All web testing was done against my own local WebGoat instance, so no real system was tested.

# 2. Task Overview

## 2.1 Task 1: Introduction to Network Security Basics

**Objective:** Understand common network threats, set up basic security measures on a small network, and monitor traffic to identify unusual activity.

**Tools and methods used:** Windows Defender Firewall, the home router's Wi-Fi security settings, and Wireshark for packet capture.

Steps taken:

1. Researched the main threat types. Viruses attach to files and need a host to spread. Worms spread across networks by themselves. Trojans look legitimate but hide malicious functions. Phishing tricks people into giving up credentials or opening malicious links, and often starts the other attacks.
2. Checked the router's Wi-Fi security. The network uses WPA2-PSK encryption, which meets the task requirement (WPA2 or WPA3).
3. Checked Windows Defender Firewall. It is turned on for both private and public network profiles, blocks incoming connections to apps not on the allowed list, and notifies me when it blocks a new app. The laptop is currently on a public network profile, so those settings apply.
4. Captured traffic in Wireshark on the Wi-Fi interface and examined DNS queries and HTTP requests.

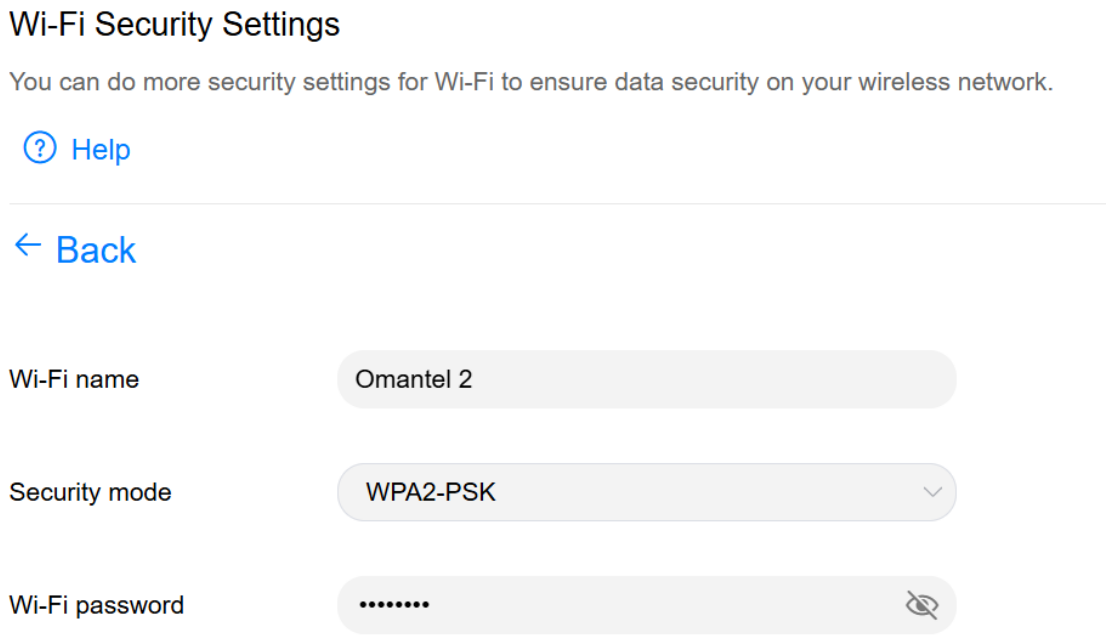



*Figure 1: Router Wi-Fi security settings showing WPA2-PSK*

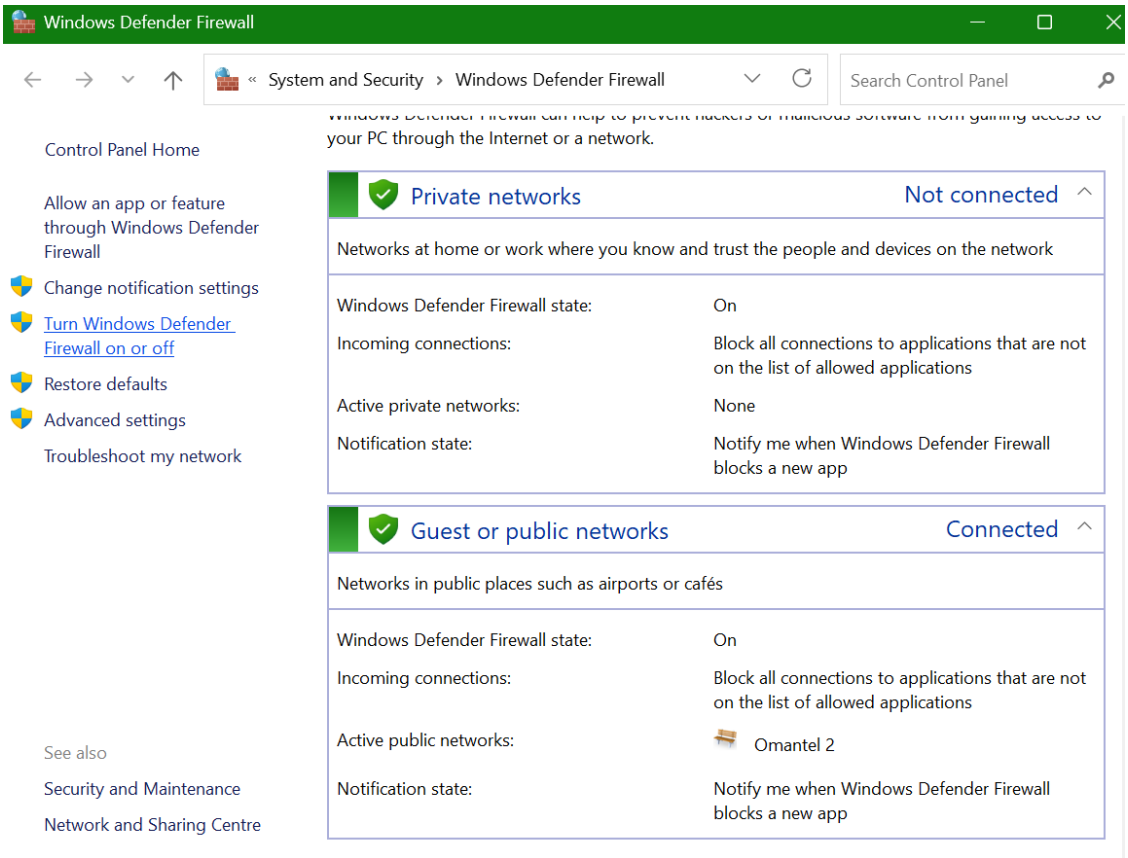



*Figure 2: Windows Defender Firewall status, public network connected and firewall on*

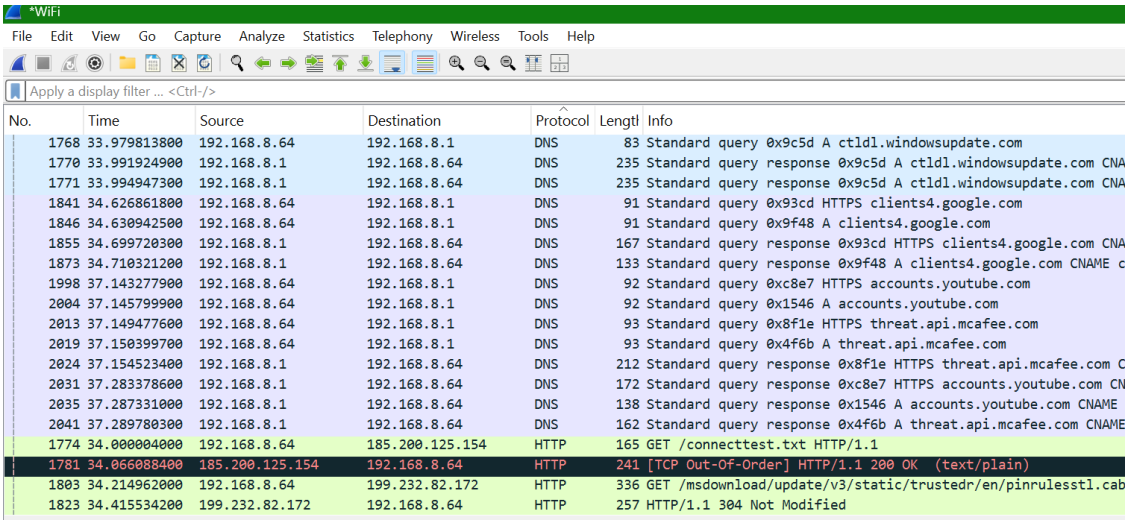



*Figure 3: Wireshark capture showing DNS queries and HTTP requests*

**Challenges and how they were overcome:** The Wireshark capture included many DNS and background requests, which made it hard to identify the important ones. I filtered by protocol (DNS and HTTP) to separate them. I also noticed an unencrypted HTTP request to /connecttest.txt on 185.200.125.154. I have not yet checked who owns that IP address, so I have not treated it as safe or unsafe.

**Results and outcomes:** The Wi-Fi uses WPA2 encryption, and the firewall is on for both network profiles. The capture showed normal DNS lookups and Windows update traffic, along with one unencrypted HTTP request that I plan to investigate further. I learned that unencrypted traffic can be read by anyone on the network, and that encryption and a firewall are basic protections against that.

## 2.2 Task 2: Introduction to Web Application Security

**Objective:** Identify common web application vulnerabilities (SQL injection, cross-site scripting, and cross-site request forgery) in a deliberately vulnerable application, exploit them manually, and understand how to prevent them.

**Tools and methods used:** WebGoat (run locally on port 8080), OWASP ZAP 2.17.0 as an HTTP proxy on port 8081, Burp Suite Community Edition, and Microsoft Edge.

Steps taken:

5. Set up the proxy. ZAP listened on 127.0.0.1:8081. I started Microsoft Edge with a separate profile and the proxy set, using: start msedge --user-data-dir="%TEMP%\zap_edge" --proxy-server="127.0.0.1:8081" --proxy-bypass-list="<-local>" "http://localtest.me:8080/WebGoat/login". I used localtest.me instead of 127.0.0.1 so that the traffic went through ZAP.
6. Created a test account in WebGoat and confirmed that localtest.me:8080 appeared under ZAP's Sites tree and in its History log.

Vulnerability 1: SQL injection.

7. In the SQL injection (intro) lesson, I ran the query SELECT department FROM employees WHERE first_name = 'Bob'; and got the result "Marketing".
8. In the String SQL injection exercise, I entered ' or '1' = '1 into the last-name field. The condition is always true, so the query returned every row in the user table.
9. In ZAP, I filtered the History tab to POST requests and found the requests to the SqlInjection endpoints. The request body held the URL-encoded SQL payload, and the response was 200 OK with lessonCompleted set to true.

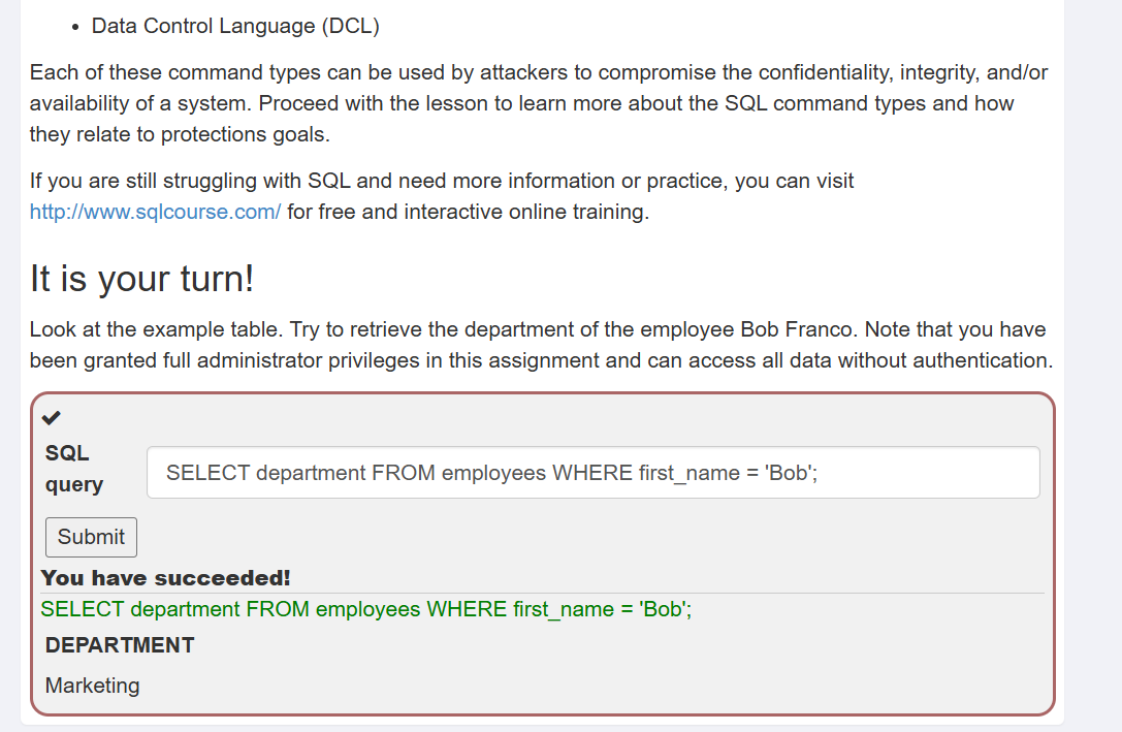



*Figure 4: SQL injection intro lesson, department query succeeded*

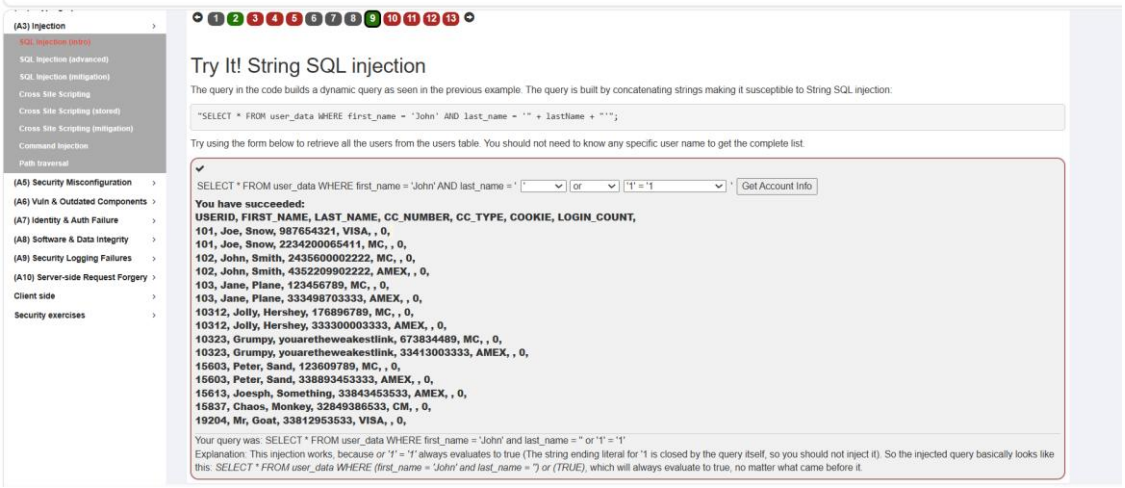



*Figure 5: String SQL injection exercise, full user table returned*

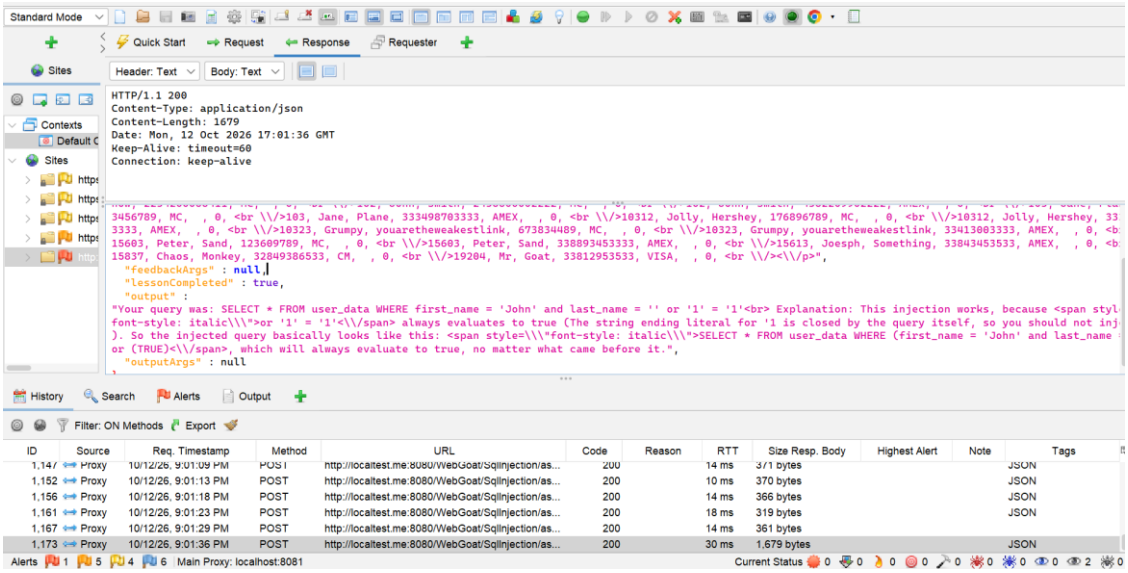



*Figure 6: Intercepted response showing the injected query and lessonCompleted: true*

Vulnerability 2: Cross-site scripting (XSS).

10. In the Try It! Reflected XSS exercise, I entered <script>alert()</script> into the credit card number field and clicked Purchase. The alert ran, and the lesson confirmed the exercise was complete.

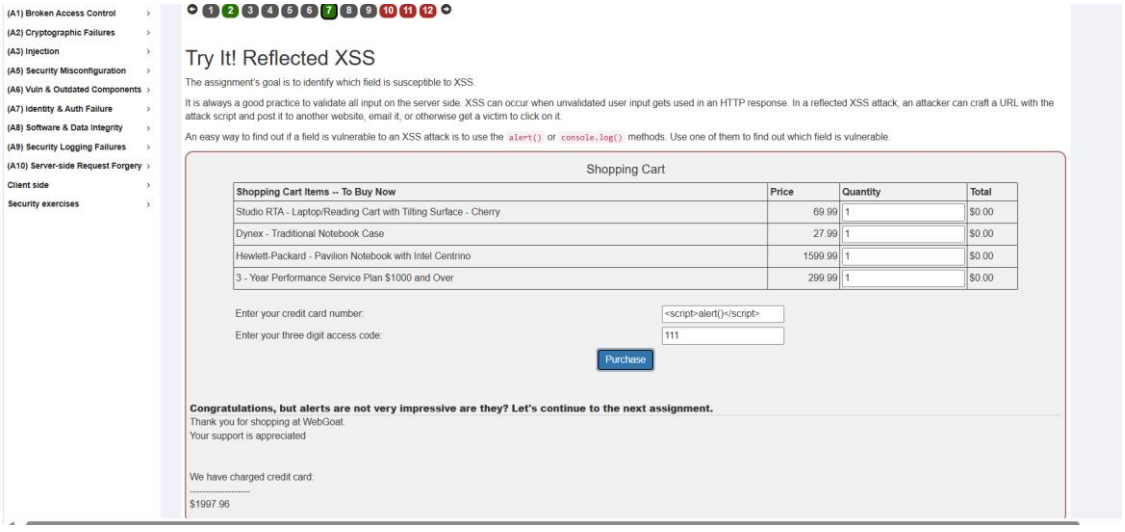


*Figure 7: Reflected XSS payload executed in the credit card number field*

Vulnerability 3: Cross-site request forgery (CSRF).

11. In the Cross-Site Request Forgeries lesson (Basic Get CSRF exercise), I submitted the form in the browser. WebGoat rejected it with "Appears the request came from the original host."
12. Sent the request from a separate host using Burp Suite with the csrf=false parameter. WebGoat returned flag 36040 and the message "Appears you made the request from a separate host." I entered 36040 in the Confirm Flag box and it was accepted.
13. In ZAP, I sent the captured request to the Requester tab, kept the session cookie, and changed the Referer header to https://attacker.com. WebGoat returned a null flag, success set to false, and the message "Appears the request came from the original host."

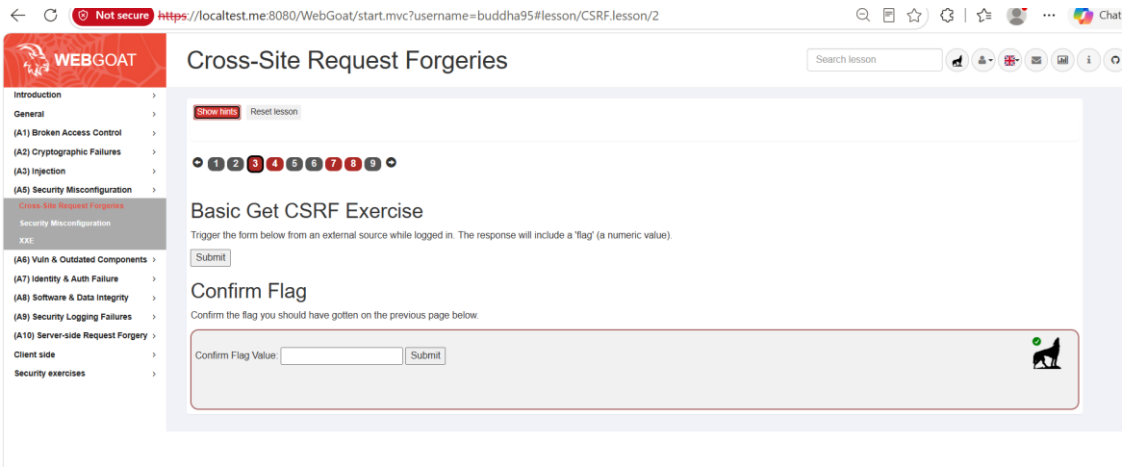



*Figure 8: Basic Get CSRF exercise in WebGoat*

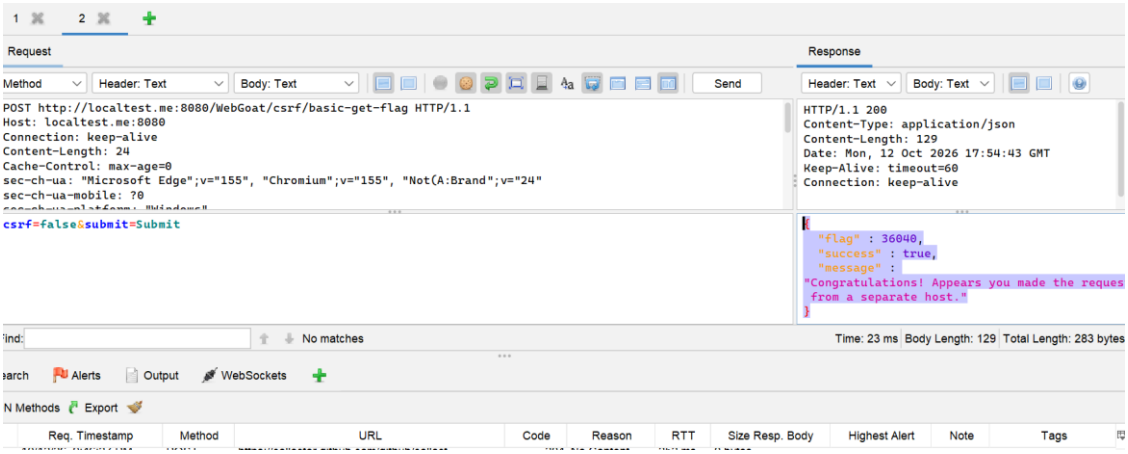



*Figure 9: Burp response returning flag 36040 for a request from a separate host*

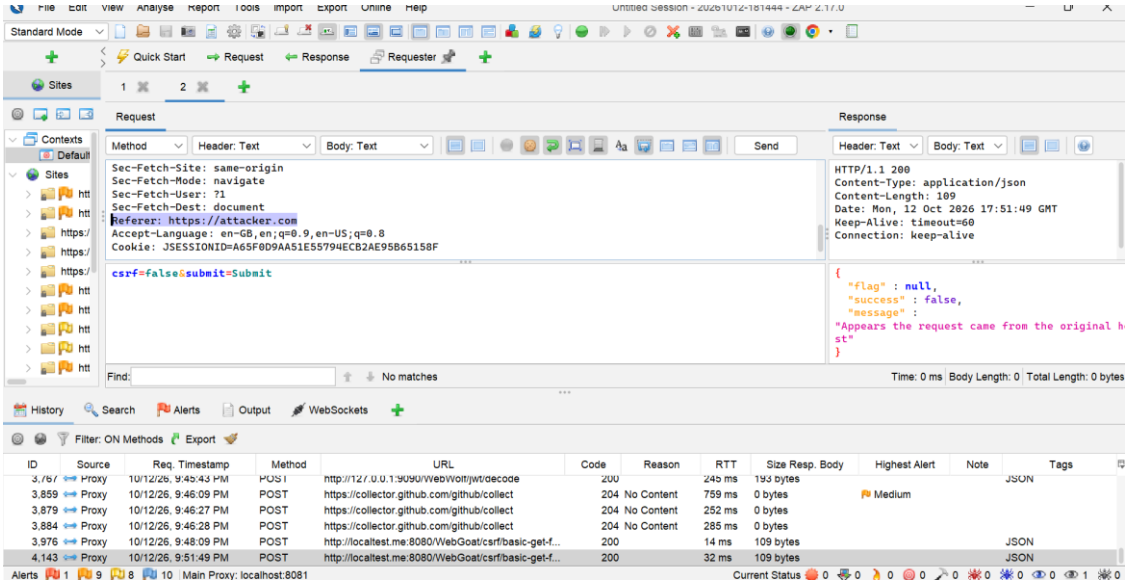



*Figure 10: ZAP request with Referer attacker.com, rejected by WebGoat*

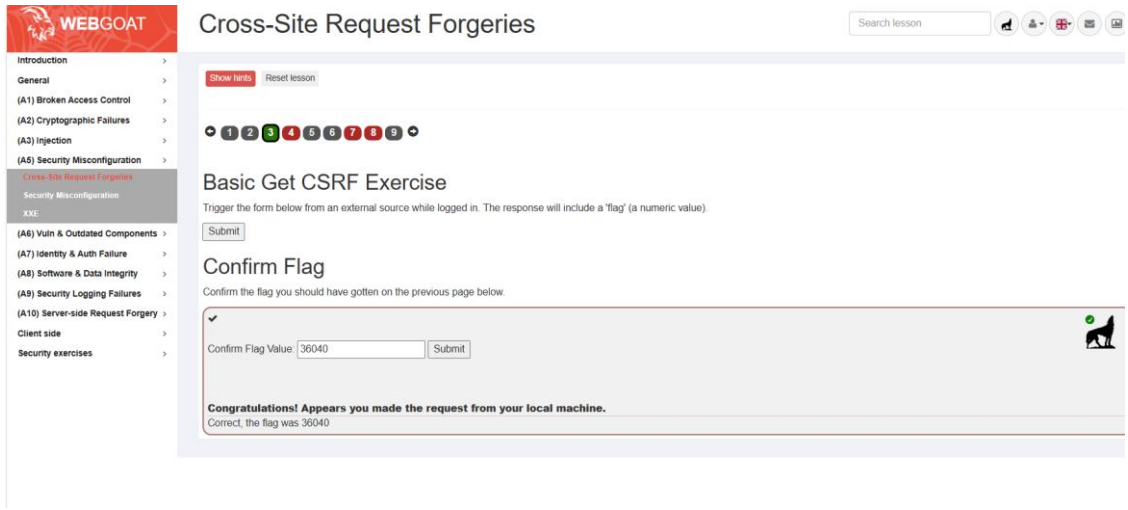



*Figure 11: Flag 36040 accepted in the Confirm Flag step*

**Challenges and how they were overcome:** The CSRF exercise rejected my first attempt because WebGoat checks where a request comes from. Replaying the request with a spoofed Referer header in ZAP did not succeed, but a request sent from a separate host did. I compared the two responses in ZAP and Burp to see the difference. The Edge proxy set-up also needed care: I used localtest.me and a separate browser profile so that the traffic reached ZAP.

**Results and outcomes:** Three vulnerabilities were demonstrated: SQL injection returned all user records; reflected XSS ran a script in the purchase form; and the CSRF exercise returned the flag from a cross-origin request. Each exercise was completed in WebGoat. The most serious finding is SQL injection, because it exposed every row of the user table in this lab. Mitigations are parameterised queries for SQL injection, output encoding and a Content Security Policy for XSS, and anti-CSRF tokens plus SameSite cookies for CSRF.

# 3. Conclusion

The internship gave me hands-on experience with both network and web security. Task 1 taught me how a basic home network is protected and how traffic can be observed. Task 2 showed me how small coding mistakes, such as building SQL queries from user input or trusting a request's origin, can expose data. I also learned to use ZAP and Burp together, to read requests and responses, and to match each attack to a fix.

For future interns, I recommend learning the proxy set-up early, because most of the evidence comes from request history. Keep screenshots and notes as you work, and check that each screenshot matches what it is supposed to show.

# 4. Closing Remarks

Thank you to the Redynox team for the tasks and guidance. The task guidelines were clear and the progression from network basics to web testing made sense. One suggestion for the programme is to provide sample responses or expected results for the WebGoat exercises, so interns can check their work against them.